Algoritmos Avançados

Aula 17: Algoritmos de Busca em Grafos (BFS e DFS)

Prof. Me. Douglas Samuel Gonçalves

Fatec Bebedouro – Jorge Caram Sabbag
Tecnologia em Big Data no Agronegócio

2º Semestre / 2026

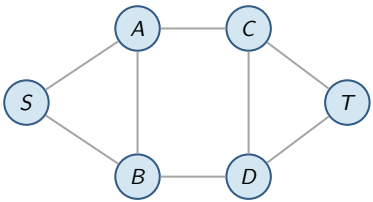
Sumário da Aula

1. Motivação & Fundamentos da Travessia em Grafos
2. Busca em Largura (BFS – Breadth-First Search)
3. Busca em Profundidade (DFS – Depth-First Search)
4. Comparativo Rigoroso: BFS vs. DFS
5. Implementação Orientada a Objetos em Python
6. Estudos de Caso Práticos em Big Data & Agronegócio
7. Exercícios Práticos Resolvidos & Desafios
8. Próximos Passos & Encerramento

O Desafio Central: Por Que Precisamos de Algoritmos de Busca?

Diferença Fundamental entre Estruturas:

- **Vetores e Listas:** Ordem linear natural ($i \rightarrow i+1$);
- **Árvores:** Raiz única, caminho único até qualquer folha, acíclicas por definição;
- **Grafos Gerais:**
  - Não possuem nó inicial canônico;
  - Podem existir múltiplos caminhos entre dois nós;
  - Presença de **ciclos**: perigo iminente de loops infinitos sem controle de visitação!



Objetivo Primordial de uma Travessia

Visitar sistematicamente cada vértice e aresta de um grafo de forma controlada, garantindo processamento em tempo linear $O(V+E)$ sem redundâncias.

Perguntas Críticas

- O nó T é alcançável a partir de S?
- Qual o menor caminho entre S e T?
- O grafo contém dependências cíclicas?

O Mecanismo Fundamental: O Paradigma das Três Cores

Para evitar reprocessamento e ciclos infinitos, a literatura clássica (Cormen et al.) modela o estado de cada vértice através de **três cores**:

BRANCO (White)



- **Estado:** Não visitado;
- Ainda não descoberto pelo algoritmo;
- $d[u] = \infty$, $\pi[u] = \text{None}$;
- Estado inicial de todos os vértices em V.

CINZA (Gray)



- **Estado:** Descoberto / Aberto;
- Na fronteira de exploração;
- Presente na **Fila** (BFS) ou na **Pilha** (DFS);
- Possui vizinhos ainda brancos a examinar.

PRETO (Black)



- **Estado:** Finalizado / Fechado;
- Totalmente explorado;
- Todos os seus vizinhos já foram visitados;
- Retirado da fronteira ativa de busca.

Dois Paradigmas Opostos: Largura (BFS) vs. Profundidade (DFS)

A estratégia adotada para gerenciar os nós **cinzas** (a fronteira de exploração) define a natureza do algoritmo:

Busca em Largura (BFS)

- **Estrutura Auxiliar:** Fila FIFO (*First-In, First-Out*);
- **Estratégia:** Explora a vizinhança em **ondas concêntricas**;
- Visita todos os nós a distância k antes de avançar para a distância $k + 1$;
- **Metáfora:** Propagação de uma gota de água em um lago calmo;
- **Garantia Ótima:** Encontra o **menor caminho** em número de arestas (saltos).

Busca em Profundidade (DFS)

- **Estrutura Auxiliar:** Pilha LIFO (*Last-In, First-Out*) ou **Recursão**;
- **Estratégia:** Mergulha o mais fundo possível ao longo de cada ramo antes de retroceder (*backtracking*);
- Explora caminhos até as folhas antes de examinar alternativas;
- **Metáfora:** Fio de Ariadne em um labirinto mitológico;
- **Garantia Estrutural:** Detecta **ciclos**, componentes e define ordenação topológica.

BFS: Princípios Fundamentais e Propriedades

Definition (Busca em Largura – BFS)

Dado um grafo $G = (V, E)$ e um vértice de origem $s \in V$, o algoritmo BFS explora sistematicamente as arestas de G para descobrir todos os vértices alcançáveis a partir de s, calculando a **menor distância** (número mínimo de arestas) e gerando uma **Árvore BFS**.

Atributos Mantidos pelo Algoritmo

Para cada vértice $v \in V$:

- $\text{cor}[v] \in \{\text{BRANCO}, \text{CINZA}, \text{PRETO}\}$;
- $d[v]$: Distância mínima (número de saltos) de s até v;
- $\pi[v]$: Vértice predecessor de v na Árvore de Busca (usado para reconstruir o caminho).

Papel da Fila FIFO (Q)

- A fila mantém os vértices **cinzas** em ordem estritamente crescente de suas distâncias a s:

$$d[u] \leq d[v] \leq d[u] + 1 \quad \forall u, v \in Q$$

- Garante que nenhum vértice de nível $k+1$ seja visitado antes de todos os vértices de nível k.

Algoritmo Formal do BFS: Pseudocódigo Estruturado

Pseudocódigo: BFS(G, s)

Para cada $u \in V[G] - \{s\}$ **faça**:
  cor$[u] \leftarrow$ BRANCO; $d[u] \leftarrow \infty$; $\pi[u] \leftarrow$ NIL
cor$[s] \leftarrow$ CINZA; $d[s] \leftarrow 0$; $\pi[s] \leftarrow$ NIL
$Q \leftarrow \emptyset$
ENQUEUE(Q, s)
Enquanto $Q \neq \emptyset$ **faça**:
  $u \leftarrow$ DEQUEUE(Q)
  Para cada $v \in$ Adj$[u]$ **faça**:
    Se cor$[v] ==$ BRANCO **então**:
      cor$[v] \leftarrow$ CINZA
      $d[v] \leftarrow d[u] + 1$
      $\pi[v] \leftarrow u$
      ENQUEUE(Q, v)
  cor$[u] \leftarrow$ PRETO

Invariantes Críticos

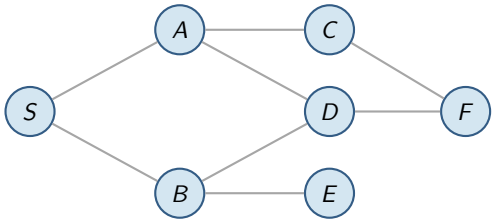
1. **Unicidade na Fila:** Cada vértice entra na fila no máximo uma vez (cor = BRANCO $\rightarrow$ CINZA).
2. **Monotonicidade:** As distâncias desenfileiradas formam uma sequência não-decrescente:

$$0, 1, 1, \ldots, 2, 2, \ldots, k, k, \ldots$$

3. **Arestas de Árvore:** As arestas $(\pi[v], v)$ formam uma floresta/árvore geradora enraizada em s.

Grafo Didático de Referência para Simulação

Utilizaremos o seguinte grafo não-direcionado com $|V| = 7$ vértices e $|E| = 8$ arestas para simular o BFS a partir da raiz S:

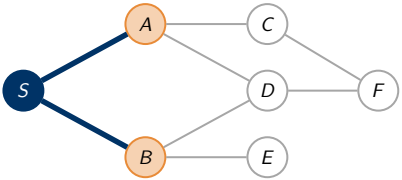


Estrutura Topológica

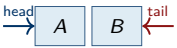
- **Vértices:** $V = \{S, A, B, C, D, E, F\}$;
- **Origem da Busca:** $s = S$;
- **Vizinhanças (ordem alfabética):**
  - $\text{Adj}[S] = \{A, B\}$
  - $\text{Adj}[A] = \{C, D, S\}$
  - $\text{Adj}[B] = \{D, E, S\}$
  - $\text{Adj}[C] = \{A, F\}$
  - $\text{Adj}[D] = \{A, B, F\}$
  - $\text{Adj}[E] = \{B\}$
  - $\text{Adj}[F] = \{C, D\}$

Rastreio Passo a Passo do BFS: Nível 0 e Nível 1

Passo 1: Descoberta dos Vizinhos de S



Estado da Fila FIFO:



Evolução do Estado

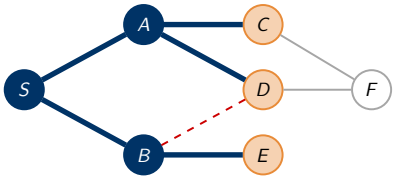
1. **Inicialização ($d = 0$):**
   - $d[S] = 0, \pi[S] = \text{None}$.
   - Fila inicial: $[S]$.
2. **Processa S:**
   - S é desenfileirado ($S \rightarrow$ PRETO).
   - Vizinhos de S: A e B (brancos).
   - $d[A] = 1, \pi[A] = S$; Enfileira A.
   - $d[B] = 1, \pi[B] = S$; Enfileira B.
   - Arestas (S, A) e (S, B) viram **Arestas de Árvore**.

Tabela Parcial de Distâncias

Vértice	S	A	B	C	D	E	F
$d[v]$	**0**	**1**	**1**	∞	∞	∞	∞
$\pi[v]$	–	S	S	–	–	–	–

Rastreio Passo a Passo do BFS: Nível 2

Passo 2: Processamento de A e B



Estado da Fila FIFO:



Evolução do Estado

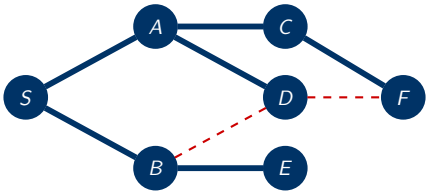
1. **Desenfileira A ($d = 1$):**
   - Vizinhos: C, D (brancos) e S (preto).
   - $d[C] = 2, \pi[C] = A$; Enfileira C.
   - $d[D] = 2, \pi[D] = A$; Enfileira D.
   - $A \rightarrow$ PRETO.
2. **Desenfileira B ($d = 1$):**
   - Vizinho D: já é CINZA! Aresta (B, D) é de **cruzamento** (não entra na árvore).
   - Vizinho E (branco): $d[E] = 2, \pi[E] = B$; Enfileira E.
   - $B \rightarrow$ PRETO.

Tabela Parcial de Distâncias

Vértice	S	A	B	C	D	E	F
$d[v]$	0	1	1	**2**	**2**	**2**	∞
$\pi[v]$	–	S	S	A	A	B	–

Rastreio Passo a Passo do BFS: Nível 3 e Árvore Final

Árvore Geradora de Menor Caminho (BFS)



Fila Final: [ ] (Vazia $\implies$ Fim do BFS)

Conclusão da Travessia

- Desenfileira C: descobre F ($d[F] = 3, \pi[F] = C$). Enfileira F.
- Desenfileira D: vizinho F já está cinza na fila (aresta de cruzamento).
- Desenfileira E e depois F: sem novos vizinhos brancos.
- **Ordem de Visitação:** $\langle S, A, B, C, D, E, F \rangle$.

Tabela Final de Distâncias e Predecessores

Vértice	S	A	B	C	D	E	F
$d[v]$	**0**	**1**	**1**	**2**	**2**	**2**	**3**
$\pi[v]$	–	S	S	A	A	B	C

Propriedade Fundamental: Menor Caminho Não-Ponderado

Teorema da Distância Mínima do BFS (Cormen et al., Cap. 22)

Seja $G = (V, E)$ um grafo direcionado ou não-direcionado, e $s \in V$. Na terminação do BFS(G, s), para todo vértice $v \in V$, o valor atribuído $d[v]$ é igual à menor distância $\delta(s, v)$ (número mínimo de arestas em qualquer caminho de s até v). Se não houver caminho, $d[v] = \infty$.

Por que a Fila FIFO Garante a Otimalidade?

- Os nós são explorados estritamente na ordem:

$$\text{Nível } 0 \implies \text{Nível } 1 \implies \cdots \implies \text{Nível } k$$

- Quando um nó v é descoberto pela primeira vez, ele é necessariamente alcançado por uma aresta proveniente de um nó com a menor distância possível $d[u]$;
- Logo, $d[v] = d[u] + 1$ é estritamente mínimo!

Aviso Crucial de Engenharia

Atenção: O BFS calcula o menor caminho **apenas em grafos não-ponderados** (onde todas as arestas têm peso uniforme $w = 1$).

- Para arestas com pesos arbitrários positivos (distância em km, tempo de trânsito em minutos), deve-se empregar o **Algoritmo de Dijkstra** (Aula 19).

Reconstrução do Caminho Mínimo via Predecessores

A partir do vetor de predecessores π, reconstrói-se a rota ótima retrocedendo do destino até a origem:

```python
def caminho_minimo_bfs(grafo, origem, destino):
    resultado = grafo.bfs(origem)
    pi = resultado['predecessores']

    if resultado['distancias'][destino] == -1:
        return [] # Destino inalcancavel

    caminho = []
    atual = destino
    while atual is not None:
        caminho.append(atual)
        if atual == origem:
            break
        atual = pi[atual]

    caminho.reverse()
    return caminho
```

Exemplo de Reconstrução: $S \to F$

Consultando o vetor de predecessores:

1. Destino $= F$;
2. $\pi[F] = C$;
3. $\pi[C] = A$;
4. $\pi[A] = S$;
5. Invertendo a sequência:

$$\mathbf{S} \to \mathbf{A} \to \mathbf{C} \to \mathbf{F}$$

6. Distância em Saltos: 3.

Complexidade de Tempo e Espaço do BFS

Complexidade de Tempo: $\mathcal{O}(|V| + |E|)$

- **Vértices:** Cada vértice entra na fila no máximo uma vez e é desenfileirado uma vez:

$$\sum_{v \in V} \mathcal{O}(1) = \mathcal{O}(|V|)$$

- **Arestas:** A lista de adjacência de cada vértice é percorrida apenas quando o vértice é desenfileirado:

$$\sum_{u \in V} |\text{Adj}[u]| = \begin{cases} |E| & \text{(Digrafos)} \\ 2|E| & \text{(Não-dir.)} \end{cases} = \mathcal{O}(|E|)$$

- **Tempo Total:** $\mathcal{O}(|\mathbf{V}| + |\mathbf{E}|)$ (Ótimo linear).

Complexidade de Espaço: $\mathcal{O}(|V|)$

- **Fila FIFO (Q):** No pior caso (grafo estrela ou denso), a fila armazena até $|V| - 1$ vértices simultaneamente $\implies \mathcal{O}(|V|)$;
- **Vetores de Controle:**
  - Cores: $\mathcal{O}(|V|)$
  - Distâncias d: $\mathcal{O}(|V|)$
  - Predecessores π: $\mathcal{O}(|V|)$
- **Espaço Adicional:** $\mathcal{O}(|\mathbf{V}|)$.

Extensões do BFS: Floresta Conexa e Multi-Source BFS

1. BFS em Grafos Desconexos

Se o grafo possuir componentes isoladas, um BFS simples a partir de s não visitará todos os nós.

- **Solução:** Loop externo iterando sobre todos os vértices $v \in V$:
  - Se cor$[v]$ == BRANCO, inicia um novo BFS a partir de v;
  - Cada execução descobre uma nova **Componente Conexa**.

2. Multi-Source BFS (Multi-Origem)

E se existirem **múltiplos pontos de partida simultâneos**?

- **Exemplo Agro:** Dispersão simultânea de pragas a partir de múltiplos talhões infectados ou múltiplos gateways em rede mesh;
- **Algoritmo:** Enfileira todas as fontes $s_1, s_2, \ldots, s_k$ com $d = 0$ no início;
- As ondas de expansão colidem suavemente, atribuindo cada nó à fonte mais próxima em tempo $\mathcal{O}(V + E)$!

DFS: Princípios Fundamentais e a Pilha de Execução

Definition (Busca em Profundidade – DFS)

O algoritmo DFS percorre o grafo explorando o mais profundamente possível ao longo de cada ramo antes de recuar (*backtracking*). Quando todos os vizinhos de um vértice já foram visitados, a busca retrocede ao vértice precedente para explorar arestas remanescentes.

A Mecânica do Backtracking

- Utiliza naturalmente uma estrutura **LIFO** (*Last-In, First-Out*);
- Implementado elegantemente via **recursão** (usando a pilha de execução da linguagem) ou com **pilha explícita**;
- Contrasta com o BFS: o BFS se expande em largura rasa; o DFS mergulha em corredores estreitos até bater na parede.

Carimbos de Tempo ($d[v]$ e $f[v]$)

O DFS mantém um cronômetro global de passos discretos $(1, 2, \ldots, 2|V|)$:

- $d[v]$: **Tempo de Descoberta** (quando v é pintado de CINZA);
- $f[v]$: **Tempo de Finalização** (quando v é pintado de PRETO);
- O intervalo $[d[v], f[v]]$ reflete a hierarquia de ancestrais na árvore!

Algoritmo Formal do DFS: Procedimento Recursivo

Procedimento Principal: DFS(G)

Para cada $u \in V[G]$ **faça**:
  cor$[u] \leftarrow$ BRANCO; $\pi[u] \leftarrow$ NIL
tempo $\leftarrow 0$
Para cada $u \in V[G]$ **faça**:
  Se cor$[u] ==$ BRANCO **então**:
    DFS-VISIT(G, u)

Procedimento Recursivo: DFS-VISIT(G, u)

tempo $\leftarrow$ tempo $+ 1$
$d[u] \leftarrow$ tempo
cor$[u] \leftarrow$ CINZA
Para cada $v \in$ Adj$[u]$ **faça**:
  Se cor$[v] ==$ BRANCO **então**:
    $\pi[v] \leftarrow u$
    DFS-VISIT(G, v)
cor$[u] \leftarrow$ PRETO
tempo $\leftarrow$ tempo $+ 1$
$f[u] \leftarrow$ tempo

Propriedade do Tempo

Cada vértice v possui exatamente dois carimbos de tempo, satisfazendo rigorosamente:

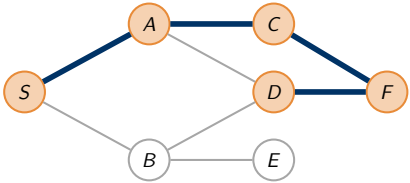
$$1 \leq d[v] < f[v] \leq 2|V|$$

Rastreio Passo a Passo do DFS: O Mergulho Profundo

Aplicando o DFS no mesmo grafo didático (ordem alfabética de vizinhos):

Etapa de Mergulho: Ramo

$S \to A \to C \to F \to D$



Pilha de Chamadas Ativa: $[S, A, C, F, D]$ (topo)

Sequência de Descobertas

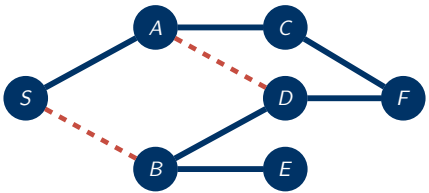
1. Inicia em S: $d[S] = 1, \text{cor}[S] = \text{CINZA}$.
2. Vizinho de S é A: $d[A] = 2, \text{cor}[A] = \text{CINZA}$.
3. Vizinho de A é C: $d[C] = 3, \text{cor}[C] = \text{CINZA}$.
4. Vizinho de C é F: $d[F] = 4, \text{cor}[F] = \text{CINZA}$.
5. Vizinho de F é D (branco):
   - A busca não retrocede em C; continua avançando para D!
   - $d[D] = 5, \text{cor}[D] = \text{CINZA}$.

Contraste com BFS

O BFS descobriu B no passo 2; o DFS já alcançou F e D sem sequer ter tocado em B!

Rastreio Passo a Passo do DFS: Backtracking e Finalização

Árvore DFS Geradora e Retorno (Finalização)



Todos os nós → **PRETO** ($f[E] = 8 \ldots f[S] = 14$).
Arestas tracejadas: **Retorno** (fecham ciclos).

Cadeia de Backtracking

- De D, visita B: $d[B] = 6, \text{cor}[B] = \text{CINZA}$.
- De B, visita E: $d[E] = 7, \text{cor}[E] = \text{CINZA}$.
- E não tem mais vizinhos brancos $\implies$ finaliza!
- $f[E] = 8, \text{cor}[E] = \text{PRETO}$.
- Desempilha para B: vizinho S já é cinza $\implies f[B] = 9$.
- Desempilha para $D \implies f[D] = 10$.
- Desempilha para $F \implies f[F] = 11$.
- Desempilha para $C \implies f[C] = 12$.
- Desempilha para $A \implies f[A] = 13$.
- Desempilha para $S \implies f[S] = 14$. Fim!

Rastreio Passo a Passo do DFS: Tabela de Tempos $[d, f]$

O resultado completo da execução do DFS no grafo com origem em S:

Atributo	**S**	**A**	**B**	**C**	**D**	**E**	**F**
Tempo Descoberta (d)	1	2	6	3	5	7	4
Tempo Finalização (f)	14	13	9	12	10	8	11
Intervalo $[d[v], f[v]]$	**[1, 14]**	**[2, 13]**	**[6, 9]**	**[3, 12]**	**[5, 10]**	**[7, 8]**	**[4, 11]**
Predecessor (π)	–	S	D	A	F	B	C

Ordem de Descoberta

$\langle S, A, C, F, D, B, E \rangle$

Ordem de Finalização (LIFO)

$\langle E, B, D, F, C, A, S \rangle$

O Teorema dos Parênteses (Parenthesis Theorem)

Teorema dos Parênteses (Cormen et al.)

Para quaisquer dois vértices u e v em uma travessia DFS de G, exatamente uma das três condições abaixo é verdadeira:

1. Os intervalos $[d[u], f[u]]$ e $[d[v], f[v]]$ são **completamente disjuntos**: nenhum é descendente do outro;
2. O intervalo $[d[u], f[u]]$ está **estritamente contido** em $[d[v], f[v]]$: u é descendente direto de v na Árvore DFS;
3. O intervalo $[d[v], f[v]]$ está **estritamente contido** em $[d[u], f[u]]$: v é descendente direto de u na Árvore DFS.

Visualização dos Intervalos Aninhados:

```
(S (A (C (F (D (B (E E) B) D) F) C) A) S)
 1  2  3  4  5  6  7 8  9 10 11 12 13 14
```

"Os tempos de descoberta e finalização têm estrutura idêntica a uma sequência bem-formada de parênteses aninhados."

Classificação das Arestas na Busca em Profundidade

Durante o DFS em um grafo direcionado, toda aresta (u, v) cai em uma de **quatro classes**:

1. Arestas de Árvore (Tree Edges)

Arestas que descobrem um vértice branco: v é visitado pela primeira vez via (u, v). Formam a floresta DFS.

2. Arestas de Retorno (Back Edges)

Conectam u a um ancestral v na árvore DFS (nó **CINZA**, ainda na pilha de recursão). **Provas cabais de ciclo no grafo!**

3. Arestas de Avanço (Forward Edges)

Arestas que conectam u a um descendente já finalizado v na árvore DFS (nó **PRETO**, com $d[u] < d[v]$).

4. Arestas de Cruzamento (Cross Edges)

Conectam nós entre ramos disjuntos da árvore ou entre árvores distintas (nó **PRETO**, com $d[u] > d[v]$).

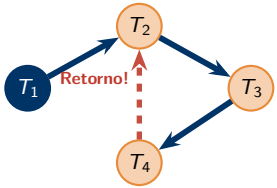
Detecção de Ciclos com DFS

Teorema Fundamental de Aciclicidade

Um grafo direcionado G é acíclico (**DAG** – *Directed Acyclic Graph*) se, e somente se, uma busca em profundidade no grafo não produzir nenhuma **Aresta de Retorno (Back Edge)**.

Algoritmo de Detecção em Digrafos:

- Ao examinar o vizinho v de u:
  - cor$[v]$ == BRANCO $\implies$ continua DFS;
  - cor$[v]$ == PRETO $\implies$ nó já finalizado (sem ciclo);
  - cor$[\mathbf{v}]$ == CINZA $\implies$ **Ciclo Detectado!** Aresta aponta para nó que ainda está na pilha de chamadas.



Em Grafos Não-Direcionados

Existe ciclo se encontrarmos um vizinho já visitado que **não seja o pai imediato** $\pi[u]$ na árvore.

DFS Iterativo com Pilha Explícita em Python

Por Que Evitar a Recursão em Big Data?

- O interpretador Python possui limite rígido de recursão (`sys.getrecursionlimit()` ≈ 1000);
- Em grafos com milhões de nós (cadeias longas), a versão recursiva dispara `RecursionError: maximum recursion depth exceeded`;
- **Solução de Engenharia:** Pilha explícita na memória Heap!

```
1  def dfs_iterativo(grafo, inicio):
2      visitados = set()
3      ordem = []
4      pilha = [inicio]
5      while pilha:
6          u = pilha.pop()
7          if u not in visitados:
8              visitados.add(u)
9              ordem.append(u)
10             for v in reversed(grafo.obter_vizinhos(
       u)):
11                 if v not in visitados:
12                     pilha.append(v)
13     return ordem
```

Mecanismo da Pilha Explícita

- `pilha.pop()` retira o último elemento inserido (semelhante ao topo da pilha de chamadas);
- Usamos `reversed(...)` para garantir que o vizinho alfabeticamente menor seja desempilhado primeiro;
- Consumo de memória restrito apenas à RAM disponível no sistema operacional.

Complexidade de Tempo e Espaço do DFS

Complexidade de Tempo: $\mathcal{O}(|V| + |E|)$

- DFS-VISIT é invocado exatamente uma vez para cada vértice $u \in V$ (quando cor$[u]$ == BRANCO);
- O laço interno examina a lista de adjacência de cada nó:

$$\sum_{u \in V} |\text{Adj}[u]| = \mathcal{O}(|E|)$$

- O custo de inicialização de todos os vértices é $\mathcal{O}(|V|)$;
- **Tempo Total:** $\mathcal{O}(|\mathbf{V}| + |\mathbf{E}|)$ (Linear).

Complexidade de Espaço: $\mathcal{O}(|V|)$

- **Pilha de Chamadas Recursivas:** Ocupa memória proporcional à profundidade máxima h do grafo:

$$\text{Memória da Pilha} = \mathcal{O}(h) \leq \mathcal{O}(|V|)$$

- Em grafos muito ramificados porém rasos, o DFS consome muito menos memória que o BFS ($\mathcal{O}(h) \ll \mathcal{O}(|V|)$);
- Tabelas auxiliares (d, f, π, cor): $\mathcal{O}(|V|)$;
- **Espaço Total:** $\mathcal{O}(|\mathbf{V}|)$.

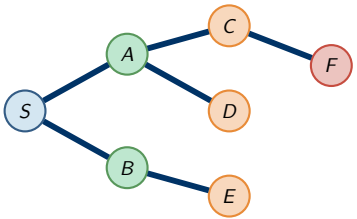
Matriz Comparativa Completa: BFS vs. DFS

Critério	Busca em Largura (BFS)	Busca em Profundidade (DFS)
Estrutura de Dados	Fila FIFO (`collections.deque`)	Pilha LIFO (`list.pop`) ou Recursão
Padrão de Exploração	Níveis concêntricos (ondas de raio r)	Ramo profundo até o fim com retrocesso
Garantia Menor Caminho	**Sim** (para grafos não-ponderados)	**Não** (apenas encontra um caminho qualquer)
Detecção de Ciclos	Arestas de cruzamento para nós no mesmo nível/nível anterior	Arestas de retorno para nós cinzas (ancestrais)
Memória no Pior Caso	$\mathcal{O}(\|V\|)$ (proporcional à largura máxima)	$\mathcal{O}(h)$ onde h é a altura máxima da árvore
Uso Ideal	Roteamento em saltos, broadcast, menor raio	Detecção de ciclos, ordenação topológica, componentes conexos
Complexidade de Tempo	$\mathcal{O}(\|V\| + \|E\|)$	$\mathcal{O}(\|V\| + \|E\|)$

Árvore BFS vs. Árvore DFS no Mesmo Grafo

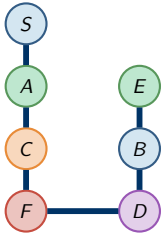
Comparação visual das árvores geradoras obtidas a partir da mesma raiz S:

Árvore BFS: Rasa e Larga



Altura da Árvore: $h = 3$.
Níveis bem delimitados por distância de saltos.

Árvore DFS: Profunda e Estreita



Altura da Árvore: $h = 6$.
Ramo linear que percorre todos os vértices em profundidade.

Guia Prático de Decisão para Engenharia de Dados

Escolha BFS se o seu objetivo for:

- Descobrir a rota de **menor número de saltos** (ex.: nós intermediários em rede de sensores);
- Identificar todos os elementos a um raio máximo K (ex.: talhões vizinhos até 20 km de um foco de praga);
- Broadcast ordenado em sistemas distribuídos;
- Buscar nós próximos à origem em grafos infinitos ou muito extensos.

Escolha DFS se o seu objetivo for:

- **Detecção de Ciclos** e Deadlocks em pipelines de tarefas (Apache Airflow, Luigi);
- **Ordenação Topológica** de pipelines de dados (Aula 18);
- Componentes Fortemente Conexos (Algoritmo de Tarjan / Kosaraju);
- Análise de Conectividade e Labirintos com menor consumo de memória em grafos muito abertos.

Arquitetura da Classe GrafoBusca em Python

Estendemos o TAD `Grafo` desenvolvido na Aula 16, adicionando os métodos de travessia:

```python
from collections import deque
from enum import Enum
from typing import Any, Dict, List, Optional, Set, Tuple

class CorDFS(Enum):
    BRANCO = 0   # Nao visitado
    CINZA = 1    # Na fronteira / Pilha de recursao
    PRETO = 2    # Totalmente explorado

class GrafoBusca(Grafo):
    """Extensao do TAD Grafo com algoritmos de busca BFS e DFS."""
    pass

```

Implementação do BFS com `collections.deque`

```python
def bfs(self, inicio: str) -> Dict[str, Any]:
    if inicio not in self._adj:
        raise KeyError(f"Vertice inicial '{inicio}' inexistente.")

    visitados = {inicio}
    distancias = {v: float('inf') for v in self.vertices}
    predecessores = {v: None for v in self.vertices}
    ordem_visita = []

    distancias[inicio] = 0
    fila = deque([inicio])  # O(1) popleft e append

    while fila:
        u = fila.popleft()
        ordem_visita.append(u)
        d_atual = int(distancias[u])

        for v in self.obter_vizinhos(u):
            if v not in visitados:
                visitados.add(v)
                distancias[v] = d_atual + 1
                predecessores[v] = u
                fila.append(v)

    return {"ordem": ordem_visita, "distancias": distancias, "predecessores": predecessores}
```

Implementação do DFS Recursivo com Tempos d/f

```python
def dfs_recursivo(self, inicio: Optional[str] = None) -> Dict[str, Any]:
    cores = {v: CorDFS.BRANCO for v in self.vertices}
    tempo_d, tempo_f, predecessores = {}, {}, {v: None for v in self.vertices}
    ordem_visita, tempo = [], 0

    def visitar(u: str):
        nonlocal tempo
        tempo += 1; tempo_d[u] = tempo; cores[u] = CorDFS.CINZA; ordem_visita.append(u)
        for v in self.obter_vizinhos(u):
            if cores[v] == CorDFS.BRANCO:
                predecessores[v] = u
                visitar(v)
        cores[u] = CorDFS.PRETO
        tempo += 1; tempo_f[u] = tempo

    lista_v = sorted(self.vertices)
    if inicio and inicio in lista_v:
        lista_v.remove(inicio); lista_v.insert(0, inicio)
    for v in lista_v:
        if cores[v] == CorDFS.BRANCO: visitar(v)

    return {"ordem": ordem_visita, "d": tempo_d, "f": tempo_f, "pi": predecessores}

```

Detecção de Ciclos em Digrafos com Recuperação da Rota

```python
def tem_ciclo_direcionado(self) -> Tuple[bool, List[str]]:
    cores = {v: CorDFS.BRANCO for v in self.vertices}
    pai = {v: None for v in self.vertices}
    ciclo = []

    def dfs_ciclo(u: str) -> bool:
        cores[u] = CorDFS.CINZA
        for v in self.obter_vizinhos(u):
            if cores[v] == CorDFS.CINZA:  # ARESTA DE RETORNO ENCONTRADA!
                ciclo.append(v)
                curr = u
                while curr != v and curr is not None:
                    ciclo.append(curr); curr = pai[curr]
                ciclo.append(v); ciclo.reverse()
                return True
            elif cores[v] == CorDFS.BRANCO:
                pai[v] = u
                if dfs_ciclo(v): return True
        cores[u] = CorDFS.PRETO
        return False

    for v in sorted(self.vertices):
        if cores[v] == CorDFS.BRANCO and dfs_ciclo(v): return True, ciclo
    return False, []

```

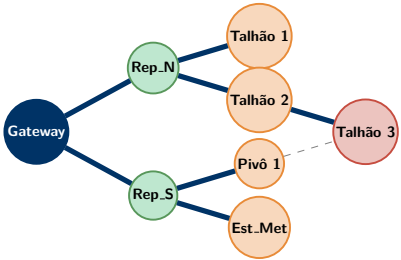
Caso 1: Roteamento de Telemetria em Rede Mesh Agrícola

Cenário Tecnológico no Campo:

- Fazenda de cana-de-açúcar monitorada por sensores de umidade do solo, pivôs centrais e estações agroclimáticas;
- Os sensores comunicam-se via protocolo sem fio **LoRaWAN em topologia Mesh**;
- Cada dispositivo só consegue transmitir diretamente para nós vizinhos no seu raio de rádio (alcance limitado);
- Os pacotes de telemetria precisam alcançar a **Usina Central (Gateway)** com o **menor consumo de bateria possível**.

Problema de Engenharia

Determinar a **árvore de roteamento de menor número de saltos (hops)** para que nenhum pacote dê voltas desnecessárias, economizando energia nos nós remotos.

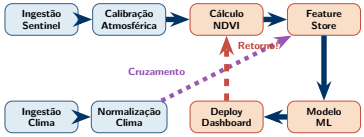


Rota Ótima para Talhão 3: 3 saltos
Gateway → Rep_N → Talhão 2 → Talhão 3

Caso 2: Deadlock em Pipelines ETL de Big Data Agro

Engenharia de Dados com Apache Airflow:

- Uma pipeline orquestra a ingestão diária de imagens de satélite Sentinel-2, dados meteorológicos e predição de produtividade;
- Cada etapa é um operador (Node); dependências são setas direcionadas (Edges);
- O Airflow exige estritamente que a pipeline seja um **DAG** (Grafo Acíclico Direcionado);
- Se um engenheiro inadvertidamente introduzir uma dependência cruzada circular, o orquestrador trava em **Deadlock**!



Ciclo Detectado:
NDVI → Feature → ML → Dash → NDVI
Aresta Pontilhada: **Cruzamento** (Clima → Feature).

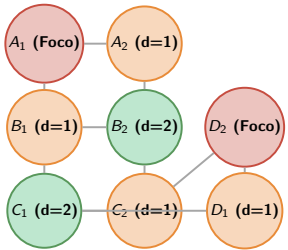
Solução via DFS

A execução do DFS com detecção de nó CINZA intercepta o ciclo antes do deploy em produção, reportando exatamente as tarefas envolvidas no deadlock.

Caso 3: Propagação Epidemiológica com Multi-Source BFS

Dispersão de Pragas (Ferrugem Asiática na Soja):

- Imagens de drones detectaram focos simultâneos de esporos nos talhões A_1 e D_2;
- A propagação ocorre para os talhões fisicamente vizinhos em ondas temporais discretas (dias de dispersão pelo vento);
- **Problema Agro:** Quais talhões devem receber pulverização emergencial nos próximos 1 e 2 dias? Qual foco original contaminou cada talhão?



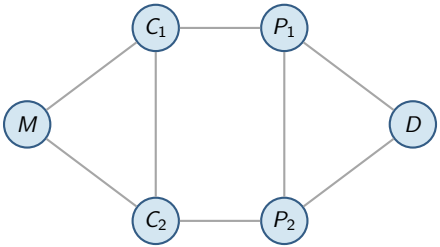
Execução Multi-Source BFS

Inicia a fila com A_1 e D_2 a $d = 0$. A expansão em ondas calcula o raio de impacto simultâneo e a bacia de atração de cada foco em tempo linear $\mathcal{O}(V + E)$.

Exercício 1 (Enunciado): Rastreio Manual de BFS

Enunciado do Exercício 1

Considere a rede de distribuição de bioinsumos entre cooperativas agrícolas representada pelo grafo não-direcionado abaixo. Execute a Busca em Largura (BFS) com origem na cooperativa **Matriz (M)**. Adote o desempate pela ordem alfabética dos vizinhos.



Pede-se:

1. Apresentar o estado da Fila FIFO em cada passo da execução;
2. Preencher a tabela com a distância em saltos (d) e o predecessor (π) de cada cooperativa;
3. Identificar o menor caminho de M até D.

Exercício 1 (Resolução Passo a Passo)

1. Evolução da Fila FIFO:

- Início: $Q = [M]$, $d[M] = 0$
- Pop M: Visita $C_1, C_2 \implies Q = [C_1, C_2]$
- Pop C_1: Visita P_1 (C_2 já na fila) $\implies Q = [C_2, P_1]$
- Pop C_2: Visita $P_2 \implies Q = [P_1, P_2]$
- Pop P_1: Visita D (P_2 já na fila) $\implies Q = [P_2, D]$
- Pop P_2: Todos vizinhos visitados $\implies Q = [D]$
- Pop D: Sem novos vizinhos $\implies Q = [\,]$

3. Menor Rota de M até D:

$$\mathbf{M \to C_1 \to P_1 \to D} \quad \text{(Distância: 3 saltos)}$$

2. Tabela de Distâncias e Predecessores:

Cooperativa	Distância (d)	Predecessor (π)
M	0	–
C_1	1	M
C_2	1	M
P_1	2	C_1
P_2	2	C_2
D	3	P_1

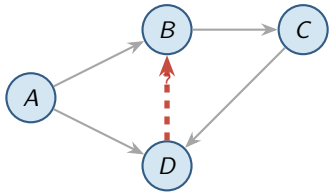
Nota Didática

A rota $M \to C_2 \to P_2 \to D$ também tem tamanho 3, mas a ordem alfabética selecionou C_1 antes de C_2 ao explorar D.

Exercício 2 (Enunciado): DFS e Detecção de Ciclos

Enunciado do Exercício 2

Considere o digrafo de dependências de processamento de satélite abaixo. Execute o DFS com início em A. Registre os carimbos de tempo de descoberta $d[u]$ e finalização $f[u]$. Classifique a aresta tracejada e determine se há ciclo.



Pede-se:

1. Os intervalos $[d[u], f[u]]$ de todos os nós;
2. A classificação da aresta (D, B);
3. Se houver ciclo, listar os vértices que o compõem.

Exercício 2 (Resolução Passo a Passo)

1. Execução do DFS:

- A descoberto: $d[A] = 1$ (CINZA)
- De $A \to B$: $d[B] = 2$ (CINZA)
- De $B \to C$: $d[C] = 3$ (CINZA)
- De $C \to D$: $d[D] = 4$ (CINZA)
- De D, examina aresta (D, B):
  - O vértice B está atualmente com a cor **CINZA** (ainda ativo na pilha de chamadas)!
  - Logo, (D, B) é uma **Aresta de Retorno (Back Edge)**.
- D finalizado: $f[D] = 5$ (PRETO)
- C finalizado: $f[C] = 6$ (PRETO)
- B finalizado: $f[B] = 7$ (PRETO)
- Aresta (A, D): D é preto, $d[A] < d[D] \implies$ **Avanço**.
- A finalizado: $f[A] = 8$ (PRETO)

2. Intervalos $[d, f]$:

Vértice	$d[u]$	$f[u]$	Intervalo
A	1	8	$[1, 8]$
B	2	7	$[2, 7]$
C	3	6	$[3, 6]$
D	4	5	$[4, 5]$

3. Ciclo Identificado

Como a aresta (D, B) é de retorno:

$$\mathbf{B} \to \mathbf{C} \to \mathbf{D} \to \mathbf{B}$$

O grafo **não é um DAG**. Há dependência circular crítica!

Exercício 3: Vulnerabilidades e Componentes Desconexos

Cenário do Problema

Um vendaval derrubou a torre de rádio repetidora em uma fazenda. O sistema de telemetria precisa identificar automaticamente se a rede fragmentou-se em ilhas isoladas de sensores que perderam o contato com a Usina.

Desafio Computacional:

- Projetar um algoritmo baseado em busca para particionar os vértices em suas **Componentes Conexas**;
- Qual algoritmo (BFS ou DFS) é mais eficiente para esta tarefa?
- Qual a complexidade assintótica final?

Solução Teórica

- **Algoritmo:**
  1. Inicializa visitados $\leftarrow \emptyset$;
  2. Para cada nó $v \in V$:
  3. Se $v \notin$ visitados, executa BFS/DFS a partir de v;
  4. Todos os nós alcançados formam uma nova componente conexa.
- **Eficiência:** Tanto BFS quanto DFS resolvem com idêntica complexidade ótima $\mathcal{O}(|V| + |E|)$.

Resumo da Aula: O Que Aprendemos Hoje?

Busca em Largura (BFS)

- Exploração em camadas usando **Fila FIFO**;
- Garante o **menor caminho em número de saltos**;
- Constrói a árvore de distâncias mínimas;
- Aplicável a telemetria mesh e raio de contaminação.

Busca em Profundidade (DFS)

- Exploração profunda usando **Pilha LIFO** / Recursão;
- Cronômetro de descoberta (d) e término (f);
- **Arestas de Retorno** provam existência de ciclos;
- Essencial para validação de DAGs e orquestração.

"Dominar BFS e DFS é ter em mãos as duas lentes fundamentais para enxergar e navegar em qualquer estrutura de rede no universo dos dados."

Próximos Passos na Disciplina

Cronograma das Próximas Aulas

- **Aula 18:** Ordenação Topológica & Componentes Fortemente Conexos:
  - Algoritmo de Kahn vs DFS com tempo de finalização descrescente ($f[v]$);
  - Algoritmo de Kosaraju-Sharir e Algoritmo de Tarjan para SCCs;
  - Orquestração e paralelismo ótimo em DAGs do Apache Spark e Airflow.
- **Aula 19:** Caminhos Mínimos em Grafos Ponderados:
  - **Algoritmo de Dijkstra** com Fila de Prioridades (Heap Binária);
  - Roteamento com fretes e distâncias rodoviárias em km no Agro.
- **Aula 20:** Árvores Geradoras Mínimas (**Prim & Kruskal**).

Atividade Prática para Casa

Executar o script `aula13_busca_grafos.py`, testar a injeção de novas arestas na malha e verificar a detecção automática de ciclos no terminal.

Referências Bibliográficas

- CORMEN, Thomas H. et al. **Algoritmos: Teoria e Prática**. 3. ed. Rio de Janeiro: Elsevier, 2012. (Capítulo 22: Algoritmos Elementares em Grafos).
- SEDGEWICK, Robert; WAYNE, Kevin. **Algorithms**. 4. ed. Upper Saddle River: Addison-Wesley, 2011. (Chapter 4: Graphs).
- BHARGAVA, Aditya Y. **Entendendo Algoritmos: Um Guia Ilustrado para Programadores e Outros Curiosos**. São Paulo: Novatec, 2017.
- WEST, Douglas B. **Introduction to Graph Theory**. 2. ed. Upper Saddle River: Prentice Hall, 2001.
- SZWARCFITER, Jayme Luiz. **Grafos e Algoritmos Computacionais**. Rio de Janeiro: Campus, 1984.
- RUSSELL, Stuart; NORVIG, Peter. **Inteligência Artificial: Uma Abordagem Moderna**. 4. ed. Rio de Janeiro: GEN LTC, 2022.

Bons Estudos!

Dúvidas ou Perguntas?

douglas.goncalves01@cps.sp.gov.br

Fatec Bebedouro – Jorge Caram Sabbag